

RIVYN: THE AI LOG INTELLIGENCE PLATFORM

Product Architecture, Engineering Milestones & Talent Acquisition Guide

Core Engineering Team • Origin: MHP Hackathon (A Porsche Company, Germany) • Version 1.0 Enterprise • September 2026

“An LLM cannot use evidence it never sees. Raw logs sit outside the model and are far too massive to paste into every prompt. Deterministic steps must retain absolute control over parsing, columnar storage, and anomaly scoring before AI generation.”

EXECUTIVE SUMMARY • NEXT-GENERATION CLOUD OBSERVABILITY

Rivyn is an enterprise-grade AI log intelligence and noise suppression platform developed to solve the \$50B crisis in cloud observability. Modern enterprise systems generate tens of gigabytes of unformatted text logs every hour. Rivyn ingests this raw deluge at **13,000+ lines/second**, compresses it by **60% to 89%** into zero-copy columnar **Apache Parquet** binary storage, and uses 3-tier machine learning to eliminate **88% to 99.9% of alert noise**. Instead of drowning Site Reliability Engineers (SREs) in thousands of duplicate alerts, Rivyn synthesizes cascading failure spikes into **100% grounded root-cause incident tickets** with exact log line and timestamp citations.

1. Who We Are: Origin Story & Engineering Philosophy

1.1 Origin at the MHP Hackathon (A Porsche Company)

Rivyn was created during the prestigious **MHP Hackathon (“Take the Money and Run”)** in Germany. MHP (a prominent technology consultancy under Porsche AG) challenged participating engineers to tackle the foundational flaw of modern enterprise monitoring: *Challenge 1: AI-Powered Observability*.

The challenge brief illustrated the core problem with a classic computer vision analogy: *“If you search a computer vision engine for ‘find me all dogs’, it should return pictures of puppies even if the caption never literally uses the word dog.”* In cloud observability, legacy log management completely fails this test. Keyword tools match tokens literally. If an engineer searches for *“brute force login attack”*, traditional grep returns nothing because the underlying operating system log reads `authentication failure for user root from 218.188.2.4`.

Furthermore, enterprise vendors (Splunk, Datadog, Elastic) charge massive recurring ingest fees to index unstructured ASCII text that is rarely queried. When an outage strikes, engineers receive hundreds of duplicate PagerDuty alerts, causing alert fatigue and high MTTR.

1.2 Our Three Non-Negotiable Engineering Tenets

- **Binary Columnar Speed Over Text Grep:** We parse unformatted log streams once upon arrival and serialize them directly into Apache Parquet with dictionary encoding. This enables zero-copy SIMD memory-mapped scans at **28+ million rows/second** directly in memory without JVM garbage collection spikes.
- **Noise Elimination Over Alert Flooding:** Individual log warnings must never trigger on-call alerts in isolation. Rivyn's 3-tier ML engine isolates genuine behavioral rarity and correlates co-occurring failures into a single actionable incident card, slashing alert noise by up to 99.9%.
- **Strict Citations Over Generative Hallucinations:** We never pipe unindexed raw logs into large language models. Every AI diagnostic claim is strictly grounded on dense vector embeddings citing exact source lines (`[Line #1 @ 08:31:02]`), paired with an offline heuristic fallback for air-gapped environments.

2. What the Product Does: Detailed Technical Architecture

Rivyn operates an end-to-end deterministic and cognitive pipeline divided into five high-throughput stages:

STAGE 01

Single-Pass Regex Drain3 Tree Ingestion (13k+ lines/sec)

Traditional log parsing relies on brittle, handcrafted regular expressions. Rivyn compiles a unified single-pass regex mask (`COMBINED_MASK`) that matches IP addresses, UUIDs, hex values, file paths, and dates in a single scan. It dynamically replaces variable tokens with wildcards `<*>`, clustering thousands of raw lines into stable semantic templates at **6,000 to 13,372 lines/second** on a single CPU core. Integrated data governance scrubs passwords, tokens, and PII before storage.

STAGE 02

Columnar Binary Storage Engine (Apache Parquet + Snappy)

Raw log text is notoriously redundant. Repeating field names like `"timestamp"`, `"service"`, and `"level"` consume up to 80% of uncompressed storage. Rivyn maps repetitive categorical strings into compact 1-byte integer dictionaries in Apache Parquet. Powered by **PyArrow**, zero-copy memory-mapped scans push boolean filters down to the byte level, achieving **60% to 89.3% storage reduction** without memory exhaustion.

STAGE 03

Multi-Tier Machine Learning Anomaly Engine

Anomalies manifest across multiple dimensions: rarity, numerical telemetry spikes, and execution state breaks. Rivyn deploys a three-tier hybrid detector:

- **Tier 1 (Statistical Rarity):** Flags templates appearing in <1% of baseline traffic.
- **Tier 2 (Isolation Forest):** Trains an ensemble of isolation trees on multi-dimensional telemetry: `[template_freq, severity_num, time_delta, params_count, burst_zscore]`.
- **Tier 3 (Markov Sequence Mining):** Models execution state transitions between templates, flagging broken sequences where intermediate steps were skipped.

STAGE 04

Topological & Temporal Incident Correlator

When core infrastructure fails, hundreds of downstream components emit cascading errors within seconds. Rivyn merges anomalies occurring within sliding 60-second temporal windows that share identical domain entities (such as block IDs `blk_38865`, hostnames, or tenant IDs), consolidating alert floods into a unified incident ticket and slashing alert noise by **88% to 99.9%**.

STAGE 05

Grounded Copilot (RAG) & Generative Incident Reasoner

Dense vector embeddings (`all-MiniLM-L6-v2`) index sliding-window log chunks for sub-millisecond semantic retrieval. OpenAI GPT-4o, Claude, or Gemini synthesize root-cause hypotheses and actionable CLI checklists strictly citing exact lines (`[Line <id> @ <timestamp>]`). If cloud APIs time out or if deployed in air-gapped environments, a deterministic rule engine provides 100% offline coverage.

3. What We Did So Far: Proven Production Benchmarks

Rivyn has undergone rigorous evaluation across 7 **heterogeneous LogHub production datasets** comprising **2,701,198 lines** of real-world infrastructure logs. All benchmarks are verifiable using our automated evaluation harness:

Dataset & Dialect	System Category	Lines	Drain Parse Speed	Storage Slashed	Noise Reduction	Triage Speedup
HDFS	Distributed Storage	1,000,000	9,984 lines/s	59.3% (2.5x)	99.90%	9,502.3x
BGL	131k-Core Supercomputer	500,000	9,190 lines/s	89.3% (9.3x)	99.81%	5,888.2x
Spark	Analytics Compute	500,000	12,223 lines/s	77.5% (4.4x)	97.93%	592.4x
Hadoop	Big Data YARN	393,431	13,372 lines/s	82.1% (5.6x)	96.54%	359.6x
OpenStack	Cloud Infrastructure	207,820	6,143 lines/s	61.3% (2.6x)	97.45%	391.8x
ZooKeeper	Distributed Consensus	74,380	12,601 lines/s	88.2% (8.5x)	99.19%	1,522.2x
Linux	OS Syslog / Auth	25,567	11,520 lines/s	66.3% (3.0x)	88.21%	105.9x

3.1 In-Depth Analysis of Benchmark Results

- **Massive Alert Noise Reduction:** On HDFS, 94,523 raw anomalous lines were consolidated into just **24 root-cause incident tickets** (99.90% noise elimination). On OpenStack, 5,379 anomalies were reduced to 137 tickets (97.45% reduction).
- **Triage Velocity Acceleration:** Against an empirical 15-second manual review baseline, automated pipeline speedups range from **105x to 9,502x** across tested workloads.
- **Extreme Columnar Compression:** On the LLNL BlueGene/L (BGL) supercomputer RAS kernel logs, raw 64.91 MB logs were compressed into **6.94 MB of Apache Parquet storage** (9.3x reduction) with sub-30ms scan times.
- **Semantic Retrieval Precision:** Queries sharing zero literal vocabulary with target logs (e.g. *“disk replica pipeline aborted”* finding `PacketResponder terminating for block`) succeeded in 100% of cases using dense embeddings, while token search returned zero matches.

3.2 Core Platform Deliverables Already Shipped

- **Asynchronous Core Backend:** Built with FastAPI, Pydantic schemas, and non-blocking I/O.
- **PyArrow Binary Storage Engine:** Columnar parquet serializer with vectorized pushdown filters.
- **Dual-Mode Frontends:** Interactive operational console (`/app`) + enterprise product marketing portal (`/`) with real-time interactive ROI calculator.
- **Automated Test Suite:** 31 unit, integration, and routing tests passing with full coverage.

4. What We Are Doing Next: 2026–2027 Engineering Roadmap

As we transition from single-node instances to distributed multi-cluster enterprise deployments, our incoming engineering division in India will directly architect and own four major technical initiatives:

INITIATIVE 01 Sub-Second Distributed Streaming Ingestion

Transitioning from batch parsing to high-throughput continuous stream processing:

- Build native stream consumers for **Apache Kafka**, **Vector**, and **FluentBit** using Rust and C++ PyArrow bindings.
- Implement sliding-window online tree clustering sustaining **>100,000 lines/second per node** with sub-200ms end-to-end incident generation latency.
- Deploy native OpenTelemetry (OTLP) gRPC and HTTP receiver endpoints.

INITIATIVE 02 Serverless Vectorized Analytical Queries (DuckDB & DataFusion)

Enabling ad-hoc exploration across petabytes of historical logs stored in cloud object storage:

- Integrate **DuckDB** and **Apache Arrow DataFusion** for serverless SQL pushdown execution directly over remote compressed Parquet files (AWS S3, Google Cloud Storage, MinIO).
- Implement **Apache Arrow Flight RPC** for ultra-low latency distributed data transmission between ingestion nodes and analytics clients.
- Enable sub-second cross-cluster filtering without maintaining dedicated index clusters.

INITIATIVE 03 On-Premise Air-Gapped Small Language Models (SLMs)

Target enterprise customers (defense, banking, healthcare) operate in strictly air-gapped environments where cloud AI APIs are prohibited:

- Fine-tune lightweight 7B and 8B parameter models (such as Llama-3, Mistral, or Qwen) on domain-specific infrastructure outage corpora.
- Quantize models using GGUF and AWQ to run locally on customer CPU clusters or single workstation GPUs with sub-second inference response times.
- Guarantee 100% private, zero-data-leakage root-cause synthesis.

INITIATIVE 04 Linux Kernel eBPF Telemetry Probes

Moving upstream from application stdout/stderr logs directly into the operating system kernel:

- Develop lightweight **eBPF** (extended Berkeley Packet Filter) probes to capture process lifecycles, TCP socket resets, packet drops, and file descriptor errors before they are ever written to disk.
- Correlate low-level kernel signals with high-level application templates for unmatched root-cause attribution.

5. Talent Acquisition Guide: Open Roles for Engineers in India

We are building our core systems and machine learning engineering division in **India**. Engineers joining this hub are founding team members with direct architectural ownership:

1. Distributed Systems & Columnar Storage Engineer

Tech Stack: Python, C++ or Rust, Apache Arrow, PyArrow, Apache Parquet, Kafka

Architect the binary columnar storage engine, streaming Parquet writers, zero-copy memory mapping, and distributed caching topologies. Optimize SIMD query pushdown kernels and build low-latency data pipelines handling millions of events.

2. Machine Learning & Anomaly Detection Engineer

Tech Stack: Python, Scikit-Learn, PyTorch, Sentence-Transformers, HuggingFace, NetworkX

Design online clustering algorithms, unsupervised multivariate anomaly detectors, and Markov state transition models. Build dense vector embedding indices and fine-tune domain-specific Small Language Models (SLMs) for local incident reasoning.

3. Systems Full-Stack & UI Engineer

Tech Stack: Modern JavaScript/TypeScript, HTML5/CSS3, WebSockets, FastAPI, Chart.js / D3

Develop low-latency operational dashboards, real-time WebSocket incident streaming boards, and visual anomaly burst inspectors capable of rendering thousands of data points smoothly at 60 FPS.

4. Cloud Infrastructure, SRE & Security Architect

Tech Stack: Kubernetes, Docker, Helm, Terraform, Linux internals, eBPF, CI/CD

Engineer Kubernetes Operators, Helm deployments, automated enterprise PII scrubbing governance, and 100% air-gapped private VPC packaging across AWS, Azure, GCP, and bare metal.

5.1 Why Join the Rivyn Core Team?

- **Genuine Computer Science Challenges:** Work on low-level memory mapping, SIMD algorithms, online stream clustering, and local SLM inference — not boilerplate CRUD APIs.
- **Founding Equity & Autonomy:** Enjoy founding engineering equity grants, transparent ownership, and the ability to influence core architecture from day one.
- **Global Engineering Culture:** Collaborate in a high-trust, zero-bureaucracy environment committed to automated testing, reproducible benchmarks, and clean software craftsmanship.

Join the Rivyn Core Team • Building the Future of Intelligent Cloud Observability

Repository: github.com/vamshidharre/TaketheMoneyandRun_Hackathon • Contact: careers@rivyn.ai